

Automated Tests and Refactoring for Testability

Day 4 Afternoon - Debugging, Exceptions, and Testing

30 min teaching + 30 min exercises

```
def test_supported_csv():
    assert
is_supported_file("records.csv")
is True

def test_unsupported_txt():
    assert
is_supported_file("records.txt")
is False
```

```
$ pytest
===== test session starts
=====
collected 10 items

test_processing.py .....
[100%]
===== 10 passed
=====
```

Goal: automate proof that small processing functions work.

BLOCK 32

Where this block fits

Block 31 used assertions. Block 32 lets a tool find and run them repeatedly.



Instruction focus

- automated testing
- test discovery
- pytest naming conventions
- passing and failing tests
- testing exceptions
- separating input, processing, and output

Practice focus

Students refactor mixed code into small functions and write tests for the processing pieces.

BLOCK 32

Why automated tests?

Manual assertions are useful, but they do not scale well.

Manual testing

Run a few examples
Look at the output
Decide whether it looks right

Automated testing

Find many tests
Run them together
Report every pass or failure

Real benefit

You can rerun the same
proof every time the
code changes.

```
$ pytest
collected 27 items

test_processing.py
.....
test_validation.py .....

27 passed
```

**Automation makes correctness
repeatable.**

What pytest does

A testing framework automates the boring parts.



Instructor framing

pytest is not magic. It is a program that imports your test files, runs test functions, and reports what happened.

The tests still come from your understanding of correct behavior.

BLOCK 32

Test discovery: names matter

pytest looks for common test file and function names.

File names

test_processing.py
processing_test.py

Function names

```
def test_normalize_status():  
    def test_valid_record():
```

```
def test_normalize_status():  
    result = normalize_status(" ACTIVE ")  
  
    assert result == "active"
```

Readable test names

A failing test name should give you
a clue about what behavior broke.

BLOCK 32

First pytest file

Put the function in one file and the tests in another.

`processing.py`

```
def normalize_status(status):  
    return status.strip().lower()
```

`test_processing.py`

```
from processing import normalize_status  
  
def test_normalize_status():  
    result = normalize_status(" ACTIVE ")  
  
    assert result == "active"
```

Key idea

Tests call your code the same way another program would call it.

BLOCK 32

Running tests

The terminal command runs the test suite.

```
$ pytest
```

Passing output

Green dots usually mean tests passed.

```
test_processing.py .....  
[100%]  
9 passed
```

Failing output

A failure shows expected and actual values.

```
E   AssertionError  
E   assert "ACTIVE" == "active"
```

A failing test is information, not a personal insult.

BLOCK 32

Multiple tests

Each test should check one clear behavior.

```
def test_active_status():  
    assert normalize_status(" ACTIVE ") == "active"  
  
def test_inactive_status():  
    assert normalize_status(" Inactive ") ==  
    "inactive"  
  
def test_supported_csv():  
    assert is_supported_file("records.csv") is True  
  
def test_unsupported_txt():  
    assert is_supported_file("records.txt") is  
False
```

Why separate?

When one behavior breaks, the failure message points to the specific rule that failed.

Avoid

One huge test that checks everything at once.

BLOCK 32

Testing exceptions

Sometimes correct behavior is rejecting bad input.

```
import pytest

def test_percentage_rejects_zero_total():

    with pytest.raises(ValueError):
        calculate_percentage(10, 0)
```

Beginner mental model
This test says: "ValueError is expected here."

If no error happens
The test fails because the bad input was accepted.

```
def calculate_percentage(completed, total):
    if total <= 0:
        raise ValueError("total must be greater than zero")

    return completed / total * 100
```

Why some functions are hard to test

Functions that mix input, processing, and output make testing harder.

```
def calculate_percentage():  
    completed = int(input("Completed: "))  
    total = int(input("Total: "))  
  
    result = completed / total * 100  
  
    print(result)
```

This function does too much
Reads input
Converts values
Calculates result
Prints output

Testing problem
To test the calculation, you now
have to simulate input and capture
printed output.

The calculation is buried inside the interaction.

BLOCK 32

Refactor for testability

Separate the calculation from the user interaction.

```
def calculate_percentage(completed,  
total):  
    return completed / total * 100
```

```
completed = int(input("Completed: "))  
total = int(input("Total: "))  
  
result = calculate_percentage(completed,  
total)  
  
print(result)
```

Now we can test

```
assert calculate_percentage(50, 100)  
    == 50
```

Design lesson

Code that is easier to test is often easier to understand and reuse.

Separation of concerns

Think in three layers: input, processing, output.



Harder to test
Input and output depend on the
environment or the user.

Easier to test
Processing functions depend on explicit
inputs and return explicit outputs.

Most tests should focus on the middle of the pipeline.

BLOCK 32

Record processor: testable architecture

Refactor the Day 3 pipeline into independently testable steps.



```
processed_records = []  
  
for record in records:  
    processed_record = process_record(record)  
    processed_records.append(processed_record)
```

Testing target
Test the small functions first. Then
test process_record() with full
dictionaries.

BLOCK 32

Guided exercises: 30-minute plan

Refactor mixed code, then test the processing functions.

0–8 min

Split input, processing,
and output

8–15 min

Create pytest file and
imports

15–22 min

Write tests for helper
functions

22–30 min

Test full record
processing

Deliverable

A program file plus a test file that can be run with pytest.

The exercise is about structure as much as syntax.

BLOCK 32

Exercise 1: refactor this function

Break the mixed function into smaller pieces.

```
def process_user_entry():
    raw = input("File name: ")
    file_name = raw.strip().lower()

    if file_name.endswith(".csv"):
        print("supported")
    elif file_name.endswith(".json"):
        print("supported")
    else:
        print("rejected")
```

Task

Create:

```
normalize_file_name()
is_supported_file()
classify_file_name()
format_file_result()
```

8 minutes

Only the processing functions need automated tests right now.

BLOCK 32

Exercise 2: create the test file

Students write pytest tests for the refactored functions.

```
from processing import (
    normalize_file_name,
    is_supported_file,
    classify_file_name,
)

def test_normalize_file_name():
    assert normalize_file_name(" REPORT.CSV ") ==
"report.csv"
```

Add tests for
CSV accepted
JSON accepted
TXT rejected
Uppercase accepted
Whitespace normalized

7 minutes

Run
\$ pytest

BLOCK 32

Exercise 3: test the record processor

Use dictionaries to test process_record() without user input.

```
def test_valid_record_becomes_ready():  
    record = {  
        "record_id": " EQ-1001 ",  
        "file_name": " REPORT.CSV ",  
        "completion": 100,  
        "error_count": 0,  
    }  
  
    result = process_record(record)  
  
    assert result["processing_status"] == "ready"
```

Pattern

Arrange

Create record dict

Act

Call process_record()

Assert

Check returned dict

8 minutes

BLOCK 32

Applied challenge: minimum test suite

Write at least ten automated tests for the refactored record processor.

Required record tests

1. Valid record becomes ready
2. Missing ID becomes rejected
3. Unsupported file type rejected
4. Minor errors produce warning
5. Too many errors rejected

Required boundary tests

6. Uppercase file extensions accepted
7. Extra whitespace normalized
8. Completion of 0 is valid
9. Completion of 100 is valid
10. Completion above 100 rejected

```
$ pytest  
  
# Goal:  
10 passed
```

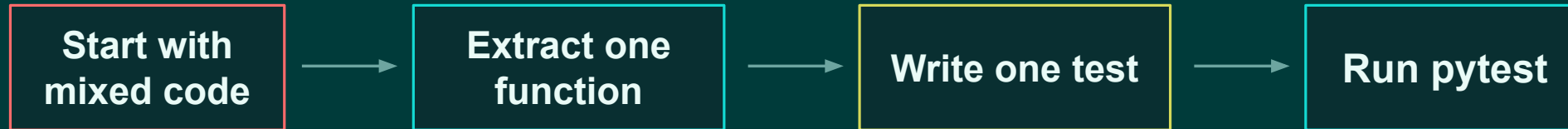
Challenge rule

Do not declare the processor complete
until the test suite passes.

BLOCK 32

Instructor demo flow

Show that refactoring and testing reinforce each other.



Teaching move

Use one small failure on purpose. Let students read the failing test and fix one cause.

The test suite becomes a safety net while changing the program.

BLOCK 32

Block 32 wrap-up

Automated tests make reliable refactoring possible.

Students can explain

- what pytest automates
- test discovery naming
- passing and failing output
- testing exceptions
- why mixed input/output code is hard to test

Students can build

- test files
- multiple test functions
- pytest exception tests
- refactored processing functions
- a 10-test record processor suite

Big idea

Well-designed functions are easier to test. Good tests make it safer to improve design.

End of Day 4: reusable pieces with evidence they work.